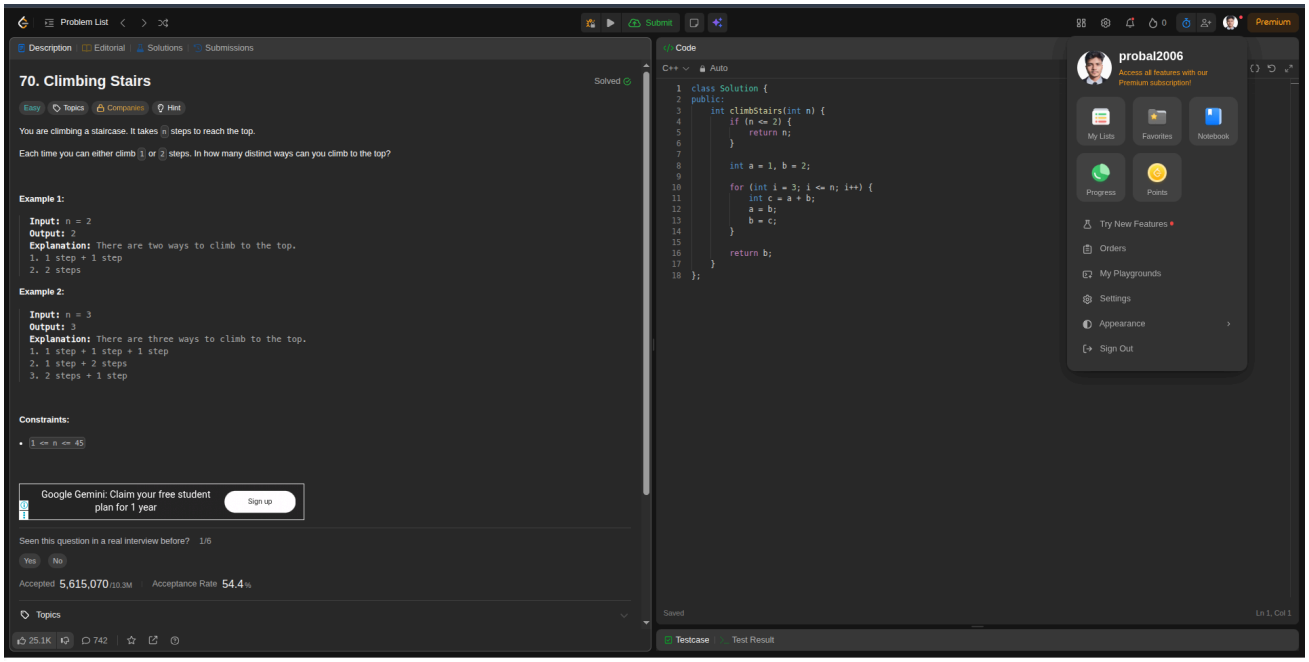
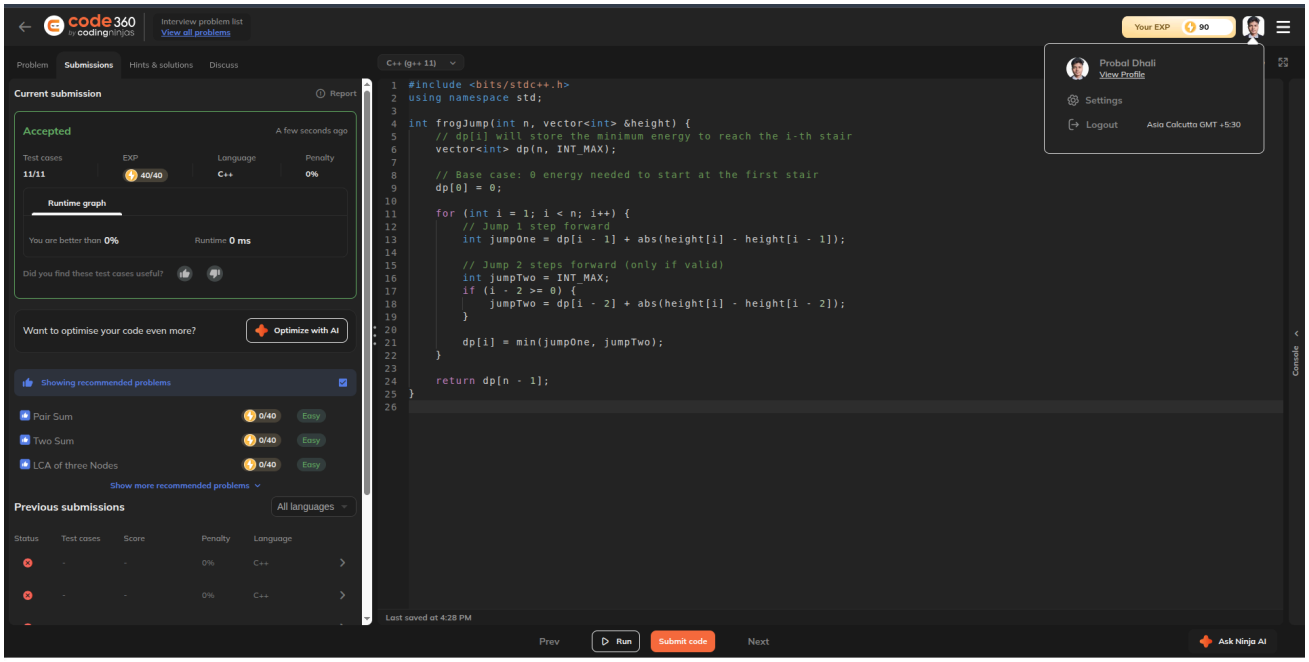

Problem List
Submit
Premium
Description
Editorial
Solutions
Submissions
70. Climbing Stairs
Solved
Easy
Topics
Companies
Hint
You are climbing a staircase. It takes n steps to reach the top.
Each time you can either climb 1 or 2 steps. In how many distinct ways can you climb to the top?
Example 1:
Input: n = 2
Output: 2
Explanation: There are two ways to climb to the top.
1. 1 step + 1 step
2. 2 steps
Example 2:
Input: n = 3
Output: 3
Explanation: There are three ways to climb to the top.
1. 1 step + 1 step + 1 step
2. 1 step + 2 steps
3. 2 steps + 1 step
Constraints:
1 <= n <= 45
Google Gemini: Claim your free student plan for 1 year
Sign up
Seen this question in a real interview before? 1/6
Yes
No
Accepted 5,615,070/10.3M
Acceptance Rate 54.4%
Topics
25.1K
742
Code
C++
Auto
class Solution {
public:
int climbStairs(int n) {
if (n <= 2) {
return n;
}
int a = 1, b = 2;
for (int i = 3; i <= n; i++) {
int c = a + b;
a = b;
b = c;
}
return b;
}
};
probal2006
Access all features with our Premium subscription!
My Lists
Favorites
Notebook
Progress
Points
Try New Features
Orders
My Playgrounds
Settings
Appearance
Sign Out
Saved
Ln 1, Col 1
Testcase
Test Result


code 360
by coding ninjas
Interview problem list
View all problems
Your EXP 90
Problem
Submissions
Hints & solutions
Discuss
C++ (g++ 11)
Probal Dhali
View Profile
Settings
Logout
Asia Calcutta GMT +5:30
Current submission
Report
Accepted
A few seconds ago
Test cases
11/11
EXP
40/40
Language
C++
Penalty
0%
Runtime graph
You are better than 0%
Runtime 0 ms
Did you find these test cases useful?
Want to optimise your code even more?
Optimize with AI
Showing recommended problems
Pair Sum
Two Sum
LCA of three Nodes
0/40
Easy
Show more recommended problems
Previous submissions
All languages
Status
Test cases
Score
Penalty
Language
#include <bits/stdc++.h>
using namespace std;
int frogJump(int n, vector<int> &height) {
// dp[i] will store the minimum energy to reach the i-th stair
vector<int> dp(n, INT_MAX);
// Base case: 0 energy needed to start at the first stair
dp[0] = 0;
for (int i = 1; i < n; i++) {
// Jump 1 step forward
int jumpOne = dp[i - 1] + abs(height[i] - height[i - 1]);
// Jump 2 steps forward (only if valid)
int jumpTwo = INT_MAX;
if (i - 2 >= 0) {
jumpTwo = dp[i - 2] + abs(height[i] - height[i - 2]);
}
dp[i] = min(jumpOne, jumpTwo);
}
return dp[n - 1];
}
Last saved at 4:28 PM
Console
Prev
Run
Submit code
Next
Ask Ninja AI